

AWESOME TANKS 2



play

AWESOME TANKS 2

with more, bigger,
longer-lasting upgrades!



play ▶

about

Music by



WATER
FLAME



\$0



performance

weapons

Select
difficulty

minigun



\$200

rocket



\$10 000

\$

\$10 000

\$28 000

\$28 000

cannon



\$10 000

mines



\$8 000

menu

stats

difficulty

play

easy ★

medium ★★

hard ★★★



\$0



performance

weapons



minigun

\$200



shotgun

\$2 750



ricochet

\$8 000



flamethrower

\$10 000



cannon

\$10 000



shock

\$10 000



rockets

\$10 000



laser

\$28 000



railgun

\$28 000



mines

\$8 000

menu

stats

difficulty

play

AWESOME TANKS 2

select level



Total score: 0 Pts.

back to upgrades

MOVE

Tap, **HOLD** and **DRAG** finger
on left side of the screen
to move



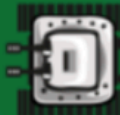
AUTO AIM

Tap, HOLD and DO NOT MOVE
your finger on the right
side of the screen



SHOOT

Tap, HOLD and MOVE your
finger on the right side
of the screen



menu





\$437



· performance ·

· weapons ·



\$2 000

ARMOR



\$500

VISIBILITY



\$500

TURRET
SPEED



\$500

MOVEMENT
SPEED

menu

stats

difficulty

play

stats.



tanks destroyed	6
towers destroyed	0
spawners destroyed	1
wallblocks destroyed	0
coins collected	56
barrels exploded	0
crates destroyed	2
total earnings	\$436

achievements





Prompts

Prompt 0

We are building an ORIGINAL browser game inspired by classic top-down tank arcade shooters. Do not create a pixel-perfect clone of any existing game, and do not recreate copyrighted art, UI, music, or level layouts exactly. Capture the FEEL instead:

- top-down tank combat
- fog-of-war exploration

- short combat-heavy levels
- destructible obstacles
- enemy bot tanks
- between-level upgrade store
- chunky military-arcade UI with green/yellow/orange tones
- satisfying explosions, recoil, screen shake, and progression

Tech stack:

- Phaser 3
- TypeScript
- Vite
- localStorage for save data
- no backend for MVP

Non-negotiable implementation rules:

- Keep code modular and readable
- Create small focused systems instead of one giant scene file
- After each milestone, run the game and fix blocking bugs before moving on
- Do not refactor unrelated stable systems unless necessary
- Maintain a PLAN.md and update it after each milestone
- Maintain a TODO.md with bugs, polish items, and future content
- Use placeholder art/SFX first unless the prompt specifically asks for polish
- Always end by reporting:
 1. what changed
 2. what works
 3. what still needs testing
 4. which files were added/modified

Prompt 1

Build the MVP of a top-down browser tank game.

Scope:

- Single playable test level using a tile/grid-based map
- Player tank can move with WASD
- Turret rotates independently to face the mouse cursor
- Left click shoots
- Player has health
- One simple enemy tank exists and can be destroyed
- Camera follows the player smoothly
- Basic HUD shows health and money
- Coins drop from destroyed enemies and can be collected
- Add a simple win condition: destroy all enemies

- Add a simple lose condition: player health reaches zero

Technical requirements:

- Separate systems for player control, enemy logic, projectiles, health, pickups, and HUD
- Use placeholder shapes/sprites first
- Add collision against walls
- Add projectile impact effects
- Keep everything deterministic and easy to tune with config values

Acceptance criteria:

- I can load the game and immediately move, aim, shoot, kill the enemy, pick up coins, and either win or die
- No console errors
- All key gameplay numbers live in config files or constants

Prompt 2

Add fog-of-war exploration to the existing game.

Design goals:

- The map starts black/unexplored
- A small visible area around the player starts revealed
- As the player moves, nearby tiles are permanently discovered
- Currently visible tiles should be brighter
- Previously explored but currently unseen tiles should remain dimly visible
- Unexplored tiles should stay black
- The reveal should feel like a top-down tactical shooter, not a soft fantasy glow

Implementation details:

- Use a performant tile-based or texture-mask approach
- Support rectangular indoor maps with corridors and rooms
- Make visibility radius upgradeable later
- Keep the system decoupled from rendering so it can be tuned easily

Acceptance criteria:

- I can move around and uncover the level gradually
- The fog stays gone where I have explored
- The game remains smooth
- The code is ready for a future “visibility” upgrade

Prompt 3

Upgrade the combat sandbox with destructible map elements and game feel.

Add:

- breakable crates
- breakable weak walls
- indestructible walls
- impact sparks
- muzzle flash
- recoil/kick on firing
- small screen shake on explosions
- debris particles on destruction
- health bars for enemies only when recently damaged

Combat tuning:

- Make the default weapon feel tight and readable
- Projectiles should move fast enough to feel responsive
- Enemy shots should be dodgeable
- Collisions must feel crisp

Level design update:

- Rework the test map so destructible geometry matters tactically
- Create at least one optional side path with a reward pickup

Acceptance criteria:

- I can blow open parts of the map
- Combat feels more satisfying than the basic prototype
- Nothing important is hidden by effects spam

Prompt 4

Add actual enemy tank AI and the real level loop.

Enemy AI requirements:

- At least 3 enemy archetypes:
 1. basic chaser/shooter
 2. slower heavy tank with higher health
 3. faster flanker with lower health
- Enemies should detect the player when the player is visible or nearby

- Use simple state-driven AI: idle, patrol, investigate, engage, reposition
- Enemies should shoot with cooldowns and imperfect aim
- Avoid suicidal pathing into walls whenever possible
- Keep behavior believable and fun, not overly smart

Level loop:

- The level ends when all enemy tanks are destroyed
- Show a clean “level complete” results screen
- Results screen displays:
 - enemies destroyed
 - coins earned
 - damage taken
 - completion time
- Then transition to a between-level shop screen

Acceptance criteria:

- The game now feels like an actual level-based tank shooter
- Enemy variety changes how I play
- The end-of-level flow works cleanly

Prompt 5

Build the between-level upgrade store and persistence.

Store requirements:

- Between levels, show a dedicated upgrade/store screen
- Player can spend coins on permanent upgrades
- Core upgrades:
 - Max Health
 - Movement Speed
 - Weapon Damage
 - Visibility Radius
- Optional bonus upgrades:
 - Fire Rate
 - Turret Rotation Speed
- Add visual bars or pips to show upgrade level
- Costs must scale nonlinearly
- Store UI should feel chunky, arcade-like, and readable

Persistence:

- Save progress in localStorage
- Save current money, unlocked weapons, purchased upgrades, current level, chosen difficulty
- Add “new game” and “continue” support

- Add a basic level select once levels are unlocked

Acceptance criteria:

- I can finish a level, enter the shop, buy upgrades, start the next level, and feel the upgrade effects immediately
- Refreshing the browser preserves progress

Prompt 6

Add a proper weapon system with unlockable weapon types.

Weapon requirements:

- Build weapons as modular data-driven configs, not hardcoded one-offs
- Add at least these weapons:
 1. machine gun
 2. shotgun
 3. cannon
 4. rockets
 5. laser
 6. mines
- Each weapon must define:
 - damage
 - fire rate
 - spread
 - projectile speed
 - splash radius
 - ammo behavior if relevant
 - recoil feel
 - VFX/SFX hooks

Controls:

- Number keys switch weapons
- Mouse wheel can cycle weapons
- HUD clearly shows selected weapon
- Locked weapons should appear in the store and unlock when purchased

Balancing:

- Machine gun = reliable starter
- Shotgun = close-range burst
- Cannon = chunky single-shot explosive
- Rockets = slower but stronger splash
- Laser = sustained beam damage
- Mines = tactical trap placement

Acceptance criteria:

- Weapon swapping feels immediate
- Every weapon changes gameplay meaningfully
- The codebase supports adding more weapons easily

Prompt 7

Expand the game into a full campaign.

Content requirements:

- Build 10 handcrafted levels first, with architecture that easily extends to 15+
- Each level needs a different layout identity:
 - narrow corridors
 - open arena
 - maze-like ambush
 - destructible shortcut map
 - heavy-tank gauntlet
 - flanker-heavy map
 - mixed final encounter
- Difficulty should ramp gradually
- Add difficulty settings: easy, medium, hard
- Difficulty should affect enemy health, aim, aggression, and rewards
- Add a level-select screen for unlocked levels
- Add mission previews or tiny level descriptors

Game flow:

- title screen
- continue/new game
- level select
- gameplay
- results
- shop
- next level

Acceptance criteria:

- The game now feels like a real campaign game, not a prototype
- Progression from level to level feels meaningful
- No level is just a copy-paste remix

Prompt 8

Do a full visual polish pass while keeping performance solid.

Art direction:

- Original top-down arcade military style
- Bold green/yellow/orange palette
- Rounded chunky buttons
- Clear readable HUD
- Distinct enemy silhouettes
- Stylized but simple, not realistic
- Strong contrast between safe UI and combat space

Polish tasks:

- Replace placeholder UI with a cohesive interface
- Add title screen branding for an ORIGINAL game name
- Add hover/press animations for buttons
- Add pause menu, restart, mute, SFX/music sliders
- Add death screen and victory screen polish
- Add achievement/stat tracking
- Add subtle camera damping and better transitions
- Ensure the fog-of-war visuals match the final art direction

Performance:

- Keep rendering efficient
- Avoid overusing particles
- Keep low-end laptops playable

Acceptance criteria:

- The game looks like a shippable indie browser prototype
- UI feels coherent across menu, store, level select, and gameplay